kmeans_shopping_mall_segmentation

May 29, 2026

[28]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
```

[29]:
```
df = pd.read_csv("/home/umair/Downloads/Mall_Customers.csv")
df
```

[29]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
..	…	…	…	…	…
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

[200 rows x 5 columns]

[30]:
```
df.isnull().sum()
```

[30]:
```
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

[31]:
```
ohe = OneHotEncoder(sparse_output=False,drop="first")
encoded_array = ohe.fit_transform(df[["Gender"]])
encoded_df = pd.DataFrame(encoded_array,columns=ohe.get_feature_names_out())
```

```
df = pd.concat([df,encoded_df],axis=1)
df.drop("Gender",axis=1,inplace=True)
df
```

[31]:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)	Gender_Male
0	1	19	15	39	1.0
1	2	21	15	81	1.0
2	3	20	16	6	0.0
3	4	23	16	77	0.0
4	5	31	17	40	0.0
..	…	…	…	…	…
195	196	35	120	79	0.0
196	197	45	126	28	0.0
197	198	32	126	74	1.0
198	199	32	137	18	1.0
199	200	30	137	83	1.0

[200 rows x 5 columns]

[32]:
```
df.drop("CustomerID",axis=1,inplace=True)
df
```

[32]:

	Age	Annual Income (k$)	Spending Score (1-100)	Gender_Male
0	19	15	39	1.0
1	21	15	81	1.0
2	20	16	6	0.0
3	23	16	77	0.0
4	31	17	40	0.0
..	…	…	…	…
195	35	120	79	0.0
196	45	126	28	0.0
197	32	126	74	1.0
198	32	137	18	1.0
199	30	137	83	1.0

[200 rows x 4 columns]

[33]:
```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
scaled_df = pd.DataFrame(scaled_data)
scaled_df
```

[33]:

	0	1	2	3
0	-1.424569	-1.738999	-0.434801	1.128152
1	-1.281035	-1.738999	1.195704	1.128152
2	-1.352802	-1.700830	-1.715913	-0.886405
3	-1.137502	-1.700830	1.040418	-0.886405

```
4   -0.563369 -1.662660 -0.395980 -0.886405
..        …       …        …        …
195 -0.276302  2.268791  1.118061 -0.886405
196  0.441365  2.497807 -0.861839 -0.886405
197 -0.491602  2.497807  0.923953  1.128152
198 -0.491602  2.917671 -1.250054  1.128152
199 -0.635135  2.917671  1.273347  1.128152

[200 rows x 4 columns]
```

1 PCA = 2 components

[34]:
```
pca = PCA(n_components=2)
final_data = pca.fit_transform(scaled_df)
final_df = pd.DataFrame(final_data)
final_df
```

[34]:
```
            0         1
0   -0.406383 -0.520714
1   -1.427673 -0.367310
2    0.050761 -1.894068
3   -1.694513 -1.631908
4   -0.313108 -1.810483
..        …       …
195 -1.179572  1.324568
196  0.672751  1.221061
197 -0.723719  2.765010
198  0.767096  2.861930
199 -1.065015  3.137256

[200 rows x 2 columns]
```

[35]:
```
pca.explained_variance_ratio_
```

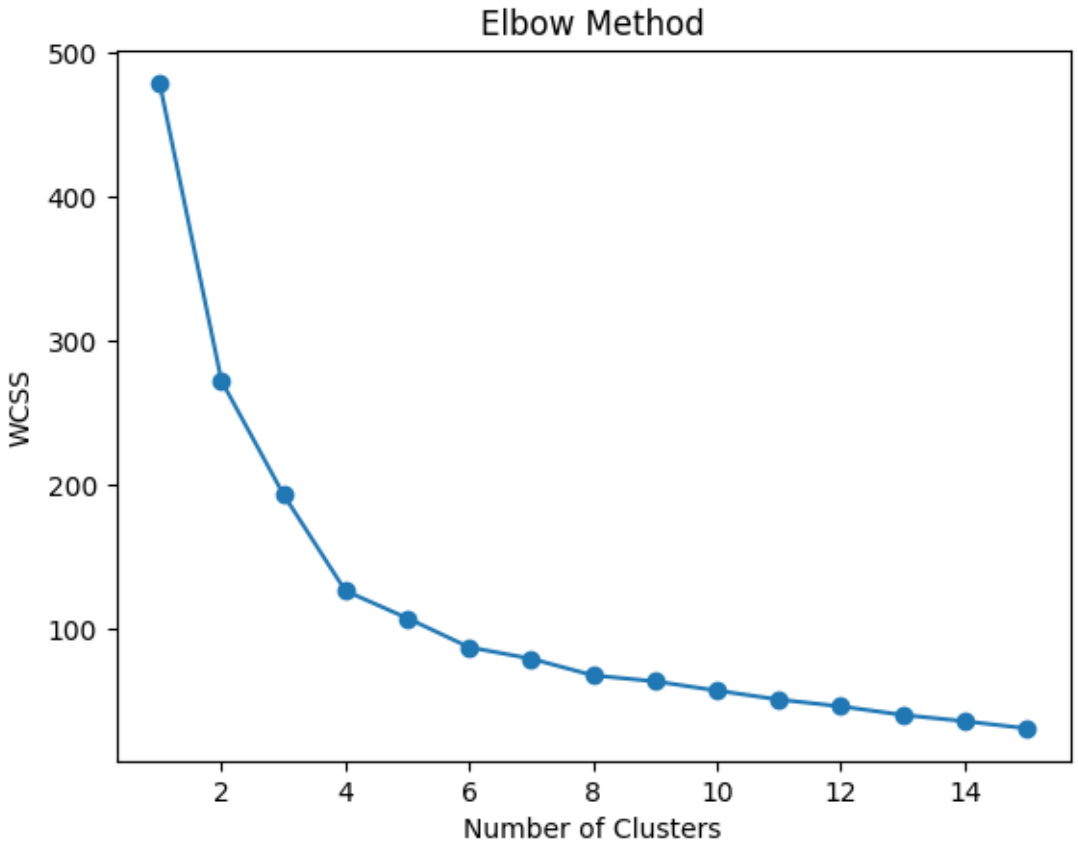
[35]:
```
array([0.33690046, 0.26230645])
```

[36]:
```
wcss = []
for k in range(1,16):
    model = KMeans(n_clusters=k,init="k-means++",random_state=42)
    model.fit(final_data)
    wcss.append(model.inertia_)
print("Done")
wcss
```

```
Done
```

[36]:
```
[479.36552158558754,
 272.16365088696875,
 193.22724083944502,
 126.25924243545263,
 107.21599102867667,
 86.94713677072878,
 79.03619176063603,
 67.41238479262904,
 63.28776768417689,
 56.85033044649647,
 50.60075343997525,
 45.86296689957007,
 39.93510477843493,
 35.542546548801816,
 30.570376688847986]
```

[37]:
```
plt.plot(range(1,16),wcss,marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```



[38]:
```
final_model = KMeans(n_clusters=4,init="k-means++",random_state=42)
```

[39]:
```
labels = final_model.fit_predict(final_data)

# Centroids
centroids = final_model.cluster_centers_

plt.figure(figsize=(9,7))

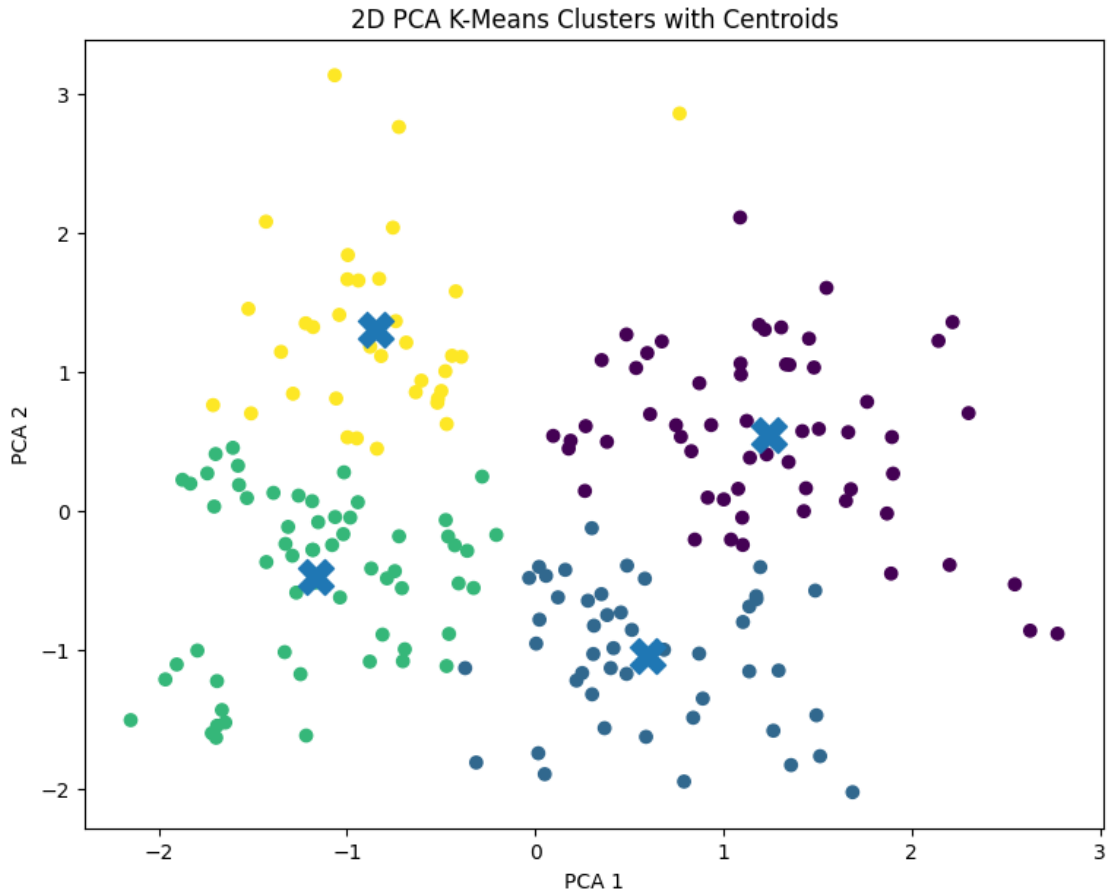
# Customer Points
plt.scatter(
    final_data[:,0],
    final_data[:,1],
    c=labels
)

# Centroid Points
plt.scatter(
    centroids[:,0],
    centroids[:,1],
    s=300,
    marker='X'
)

plt.xlabel("PCA 1")
plt.ylabel("PCA 2")

plt.title("2D PCA K-Means Clusters with Centroids")

plt.show()
```



2 PCA = 3 components

[40]:
```
pca = PCA(n_components=3)
final_data = pca.fit_transform(scaled_df)
final_df = pd.DataFrame(final_data)
final_df
```

[40]:

	0	1	2
0	-0.406383	-0.520714	2.072527
1	-1.427673	-0.367310	2.277644
2	0.050761	-1.894068	0.367375
3	-1.694513	-1.631908	0.717467
4	-0.313108	-1.810483	0.426460
..	…	…	…
195	-1.179572	1.324568	-1.932441
196	0.672751	1.221061	-2.438084
197	-0.723719	2.765010	-0.583178
198	0.767096	2.861930	-1.150341

```
199 -1.065015  3.137256 -0.788146

[200 rows x 3 columns]
```

[41]:
```
pca.explained_variance_ratio_
```

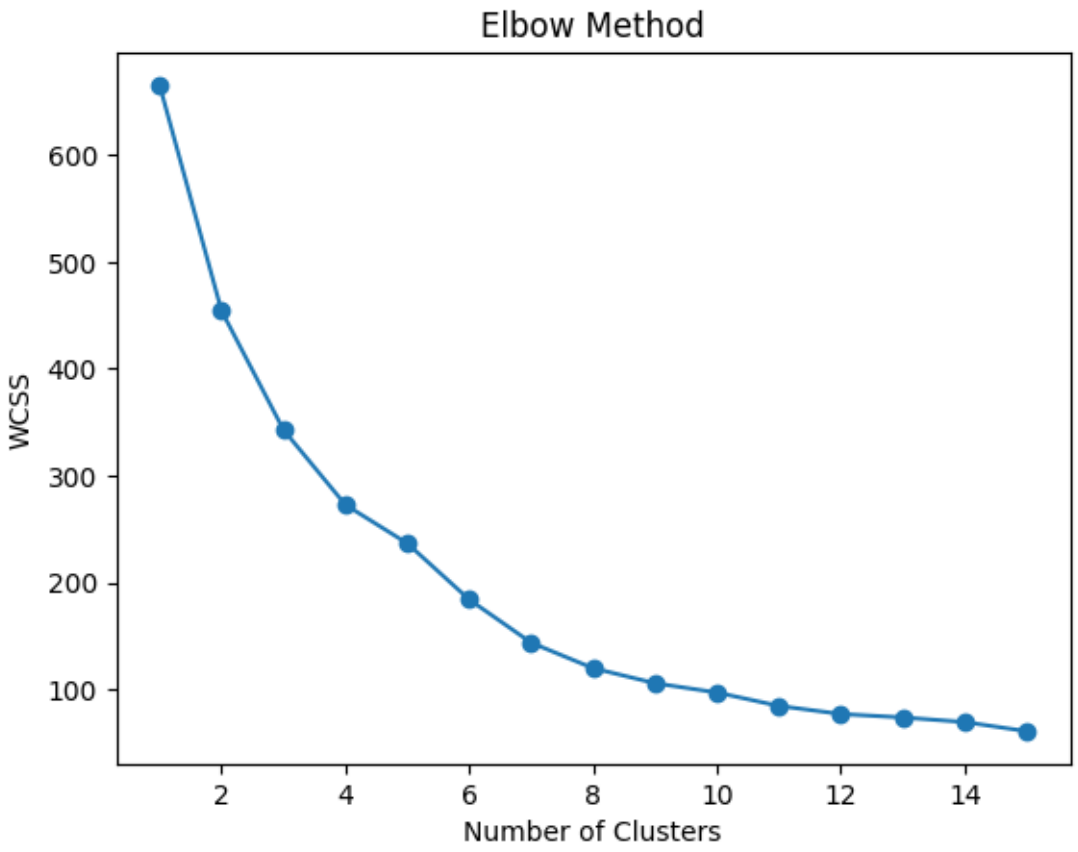
[41]:
```
array([0.33690046, 0.26230645, 0.23260639])
```

[42]:
```
wcss = []
for k in range(1,16):
    model = KMeans(n_clusters=k,init="k-means++",random_state=42)
    model.fit(final_data)
    wcss.append(model.inertia_)
print("Done")
wcss
```

```
Done
```

[42]:
```
[665.4506303076759,
 454.2650744263794,
 342.94926813183736,
 272.7860347147854,
 236.6753238993274,
 184.1127719404184,
 143.73649959327184,
 119.87907443474268,
 105.82646101450139,
 97.12895732088131,
 84.61556712269916,
 77.02883550834598,
 73.82527779267417,
 69.5021811228299,
 60.92762702449973]
```

[43]:
```
plt.plot(range(1,16),wcss,marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```



[47]:
```
final_model = KMeans(n_clusters=4,init="k-means++",random_state=42)
```

[49]:
```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(10,7))

ax = fig.add_subplot(111, projection='3d')

labels = final_model.fit_predict(final_data)

# Centroids
centroids = final_model.cluster_centers_

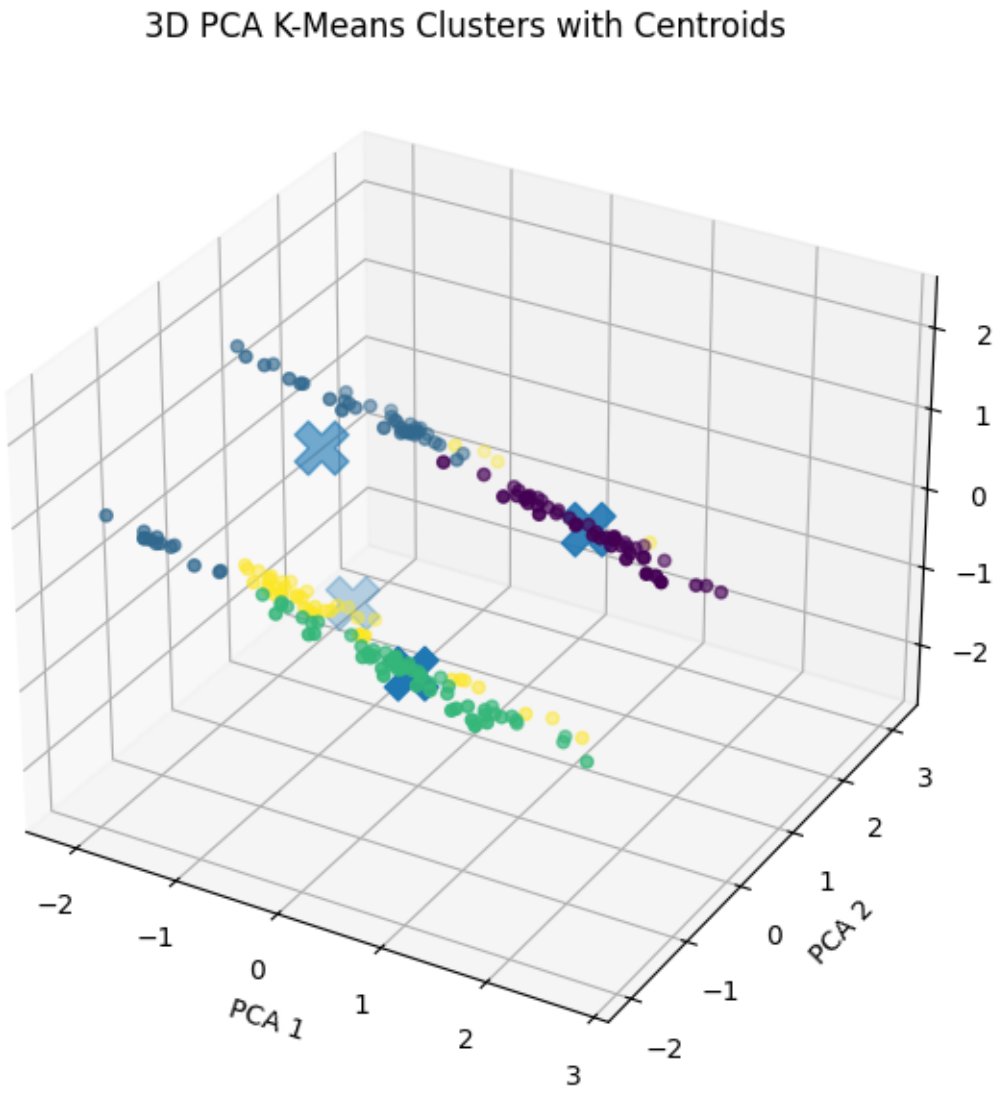
# Customer Points
ax.scatter(
    final_data[:,0],
    final_data[:,1],
    final_data[:,2],
    c=labels
)
```

```
# Centroids
ax.scatter(
    centroids[:,0],
    centroids[:,1],
    centroids[:,2],
    s=400,
    marker='X'
)

ax.set_xlabel("PCA 1")
ax.set_ylabel("PCA 2")
ax.set_zlabel("PCA 3")

plt.title("3D PCA K-Means Clusters with Centroids")

plt.show()
```


3D PCA K-Means Clusters with Centroids
PCA 1
PCA 2
−2
−1
0
1
2
3

[ ]: